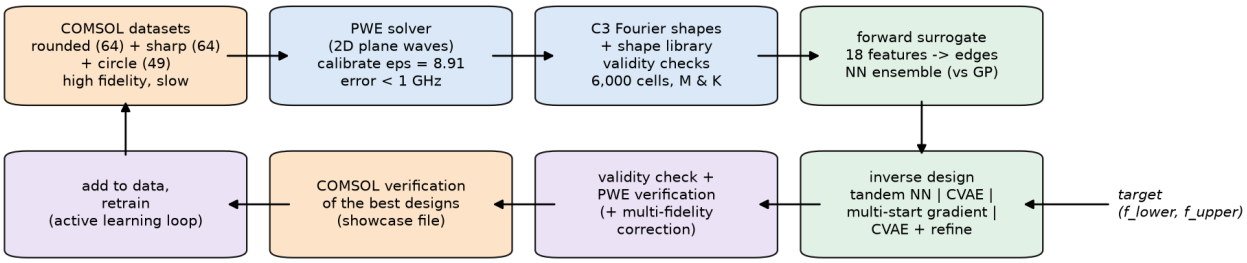
# Learning the Project from Zero

Waves, Fourier series, the plane-wave (PWE) solver and every machine-learning method in the project — step by step, with worked examples and exercises you run yourself

M.Tech project P29 — Binit Shakya, IIT Guwahati

Companion to the option-B study guide (docs/OptionB_Study_Guide.pdf)




*target band edges (f_lower, f_upper) -> several different fabricable hole shapes that give them*

*The whole project on one page. By the end of this guide you will understand every box: what it computes, how, and why.*

# Contents

**0. How to use this guide** 4

0.1 Running an exercise (do this once now) . . . 4

0.2 The exercises . . . 4

**1. The project in plain words** 5

**2. Waves and photonic crystals** 6

2.1 Frequency, wavelength, speed . . . 6

2.2 Why a periodic pattern blocks some frequencies . . . 6

2.3 The crystal and its unit cell . . . 6

2.4 The wavevector k and the Brillouin zone . . . 7

2.5 The band diagram, f_lower and f_upper . . . 7

2.6 What makes it a valley crystal . . . 7

**3. Fourier series from the beginning** 9

3.1 The idea . . . 9

3.2 The formula (real form) . . . 9

3.3 Why the formula works (orthogonality) . . . 9

3.4 Worked example: the permittivity of a layered crystal . . . 9

3.5 The complex form (used by the PWE solver) . . . 10

3.6 The FFT . . . 10

**4. Describing a hole shape with a Fourier series** 11

4.1 A hole as a function r(θ) . . . 11

4.2 Worked example: adding c3 cos 3θ . . . 11

4.3 The three-fold rule, step by step . . . 12

4.4 Nine numbers per hole, eighteen per cell . . . 12

4.5 Worked example: the old rounded triangle . . . 12

4.6 The design space and the fabrication rules . . . 12

**5. The plane-wave expansion (PWE) method in 1D** 14

5.1 Eigenvalue problems in one paragraph . . . 14

5.2 The wave equation in the crystal . . . 14

5.3 Bloch's theorem: what a wave in a crystal looks like . . . 14

5.4 Turning the equation into a matrix . . . 14

5.5 Ho's rule: invert the ε matrix . . . 15

5.6 Worked example with only 3 plane waves . . . 15

5.7 Convergence: how many plane waves? . . . 15

**6. The 2D solver of the project (src/shapes/pwe.py)** 17

6.1 Reciprocal lattice vectors . . . 17

6.2 The 2D equation . . . 17
6.3 What the code does, step by step . . . 17
6.4 Accuracy and speed . . . 17
6.5 Calibration: why ε = 8.91 and not 11.7 . . . 17
6.6 From design numbers to band edges . . . 18
**7. Machine learning: learning a function from examples 19**
7.1 What a model and training are . . . 19
7.2 Training, validation, test — and honest errors . . . 19
7.3 Neural networks . . . 19
7.4 Gaussian processes . . . 20
**8. Inverse design: running the model backwards 21**
8.1 The problem, and why it is hard . . . 21
8.2 Why the direct inverse fails . . . 21
8.3 The tandem network (Liu et al. 2018) . . . 21
8.4 Conditional variational autoencoder (CVAE) . . . 22
8.5 Multi-start gradient search . . . 23
8.6 The trap: surrogate exploitation . . . 23
8.7 CVAE + refine . . . 24
8.8 How the project compares them . . . 24
**9. Multi-fidelity: cheap simulations corrected by expensive ones 25**
**10. The whole pipeline on one example 26**
**11. Glossary 27**
**12. Formula sheet 28**
**13. Answers to the check-yourself questions 29**
Where to read more . . . 31

# 0. How to use this guide

This guide assumes nothing beyond school mathematics. Every idea comes in four steps: **the idea in words**, **a worked example with real numbers** (do it with pen and paper), **an exercise script** you run and change, and **check-yourself questions** (answers at the back, part 13). Read the parts in order — each one uses the previous ones. Parts 1–6 are the physics and the simulator, parts 7–10 the machine learning.

## 0.1 Running an exercise (do this once now)

1. Open a terminal: in VS Code use *Terminal → New Terminal*, or open *Windows Terminal / PowerShell*.
2. Go to the project folder and switch on the project's Python (the virtual environment):

```
cd "E:\MTech\MTP project\Project"
.venv\Scripts\activate          # the prompt now starts with (.venv)
python learn\ex01_waves.py
```

1. The numbers appear in the terminal. The picture is saved in `learn\figs\` — open it in VS Code or the file explorer.
2. To experiment: open the script in VS Code. The settings are the CAPITAL names near the top (for example `F_GHZ = 300.0`). Change one, save (Ctrl+S), run the same command again (press the Up arrow in the terminal to repeat it). Compare with the previous output.
3. If something breaks, undo your change (Ctrl+Z) and save. You cannot damage the project: the exercises only read it.

## 0.2 The exercises

| # | script | topic | part |
|---|---|---|---|
| 1 | ex01_waves.py | frequency, wavelength, why a = 242.5 µm | 2 |
| 2 | ex02_fourier_series.py | Fourier series of the permittivity pattern | 3 |
| 3 | ex03_hole_shapes.py | describing a hole with r(θ); the 3-fold rule; the old triangle's numbers | 4 |
| 4 | ex04_pwe_1d.py | the plane-wave method in 1D, by hand and by computer, vs the exact answer | 5 |
| 5 | ex05_pwe_2d.py | the project's real 2D solver on circular holes | 6 |
| 6 | ex06_design_to_band_edges.py | 10 design numbers → holes → fabrication check → band edges | 6 |
| 7 | ex07_surrogate_gp_vs_nn.py | learning a function: Gaussian process vs neural network | 7 |
| 8 | ex08_tandem_toy.py | why a direct inverse fails; the tandem network | 8 |
| 9 | ex09_cvae_toy.py | conditional VAE: many designs for one target | 8 |
| 10 | ex10_multistart_toy.py | gradient search from many starts; surrogate exploitation | 8 |
| 11 | ex11_multifidelity_toy.py | cheap simulator + learned correction | 9 |

`python learn\run_all.py` runs all of them and rebuilds this PDF (about 4 minutes).

# 1. The project in plain words

**The device.** A thin slab of silicon with a regular pattern of holes, the same platform as the terahertz chip of Yang et al. (Nature Photonics 2020). Terahertz waves around 300 GHz (wavelength about 1 mm in air) travel inside the slab. The pattern repeats every **a = 242.5 µm** and each repeating tile (the *unit cell*) contains **two holes**, A and B.

**What the pattern does.** For a certain range of frequencies, the *band gap*, waves cannot travel through the patterned silicon at all. If holes A and B are different, the crystal becomes a *valley photonic crystal*: at a boundary between two such crystals (one with A big, one with B big) a wave can travel along the boundary, even around sharp corners, without bouncing back. That is useful for 6G on-chip links. The band gap is described by two numbers: its bottom edge $\mathbf{f_{lower}}$ and its top edge $\mathbf{f_{upper}}$.

**The forward problem** (easy, done by simulation): given the hole shapes, compute $f_{lower}$ and $f_{upper}$. COMSOL does this accurately but slowly; our own PWE solver does it in 0.2 seconds.

**The inverse problem** (the goal of the project): given the $f_{lower}$ and $f_{upper}$ you want, find hole shapes that produce them. Simulators cannot run backwards, so we train machine-learning models to do it. In option B the holes are not limited to triangles: any shape with three-fold symmetry is allowed, described by a Fourier series (part 4). Then **many different shapes give the same band edges**, which makes the inverse problem hard and interesting (part 8).

**The chain of ideas in this guide**

waves and band gaps (2) → Fourier series (3) → hole shapes as Fourier series (4) → the PWE solver in 1D (5) and 2D (6) → learning a function from examples (7) → running it backwards: tandem, CVAE, multi-start (8) → combining cheap and expensive simulations (9) → the full pipeline on one example (10).

# 2. Waves and photonic crystals

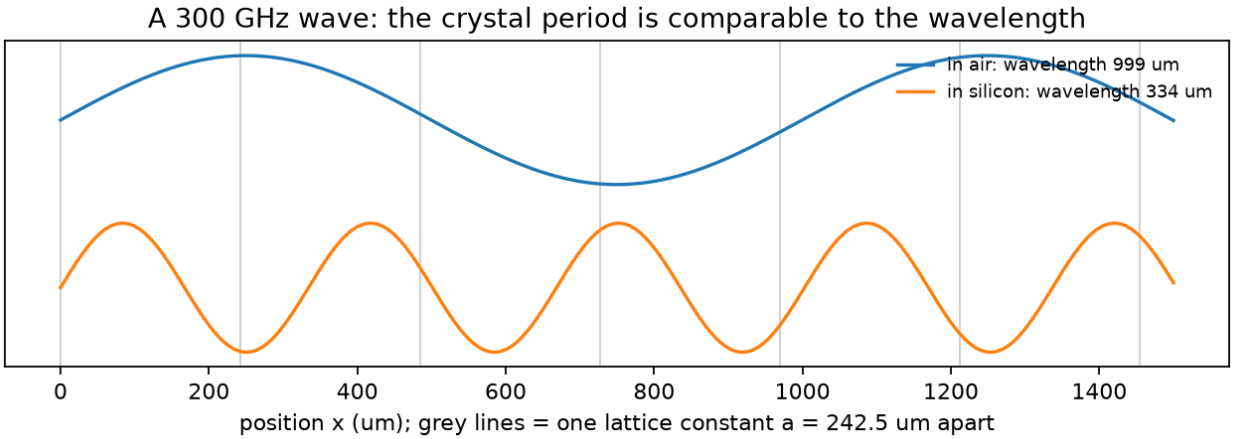
## 2.1 Frequency, wavelength, speed

A wave repeats in time and in space. **Frequency f** = how many times per second it repeats (unit Hz; 1 GHz = $10^9$ Hz). **Wavelength λ** = the distance between two crests. They are tied together by the speed of the wave:

$$c = f\lambda \qquad \Rightarrow \qquad \lambda = \frac{c}{f}$$

**Worked example.** f = 300 GHz = $3\times10^{11}$ Hz, c = $2.998\times10^8$ m/s. λ = $2.998\times10^8$ / $3\times10^{11}$ = $9.99\times10^{-4}$ m ≈ **1.0 mm = 999 µm**.

Inside a material light is slower by the **refractive index n**: speed c/n, wavelength λ/n. The **permittivity** ε (how strongly the material responds to an electric field) is ε = $n^2$. Bulk silicon: n = 3.42, ε = 11.7. In this project's 2D model the effective values are n ≈ 2.99, ε ≈ 8.91 (part 6.5). With n = 2.99: λ in silicon = 999 / 2.99 = **334 µm**.



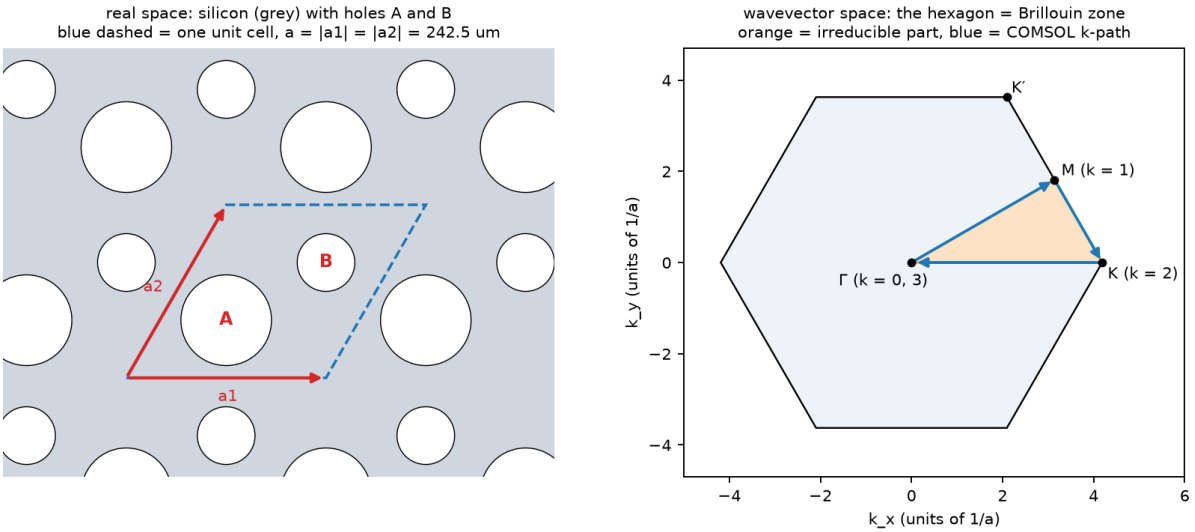

*Exercise 1: a 300 GHz wave in air and in silicon. The grey lines are 242.5 µm apart — the crystal period is about a quarter of the air wavelength.*

The ratio a/λ is called the **normalised frequency**: 242.5 / 999 = 0.243. Photonic crystals do interesting things when a/λ is roughly 0.2–0.5, i.e. when the pattern is comparable to the wavelength. That is why a = 242.5 µm was chosen for 300 GHz. Scaling the whole crystal by 2 divides every frequency by 2.

## 2.2 Why a periodic pattern blocks some frequencies

Every boundary between silicon and air reflects a little of the wave. In a periodic structure there are many boundaries, equally spaced. At most frequencies the small reflections arrive with random phases and cancel. At special frequencies — when the spacing is about half a wavelength inside the material — all reflections arrive **in step** and add up to a total reflection. The wave cannot enter: that frequency range is a **band gap**. This is the same effect that makes soap bubbles and butterfly wings coloured (Bragg reflection).

## 2.3 The crystal and its unit cell




*Left: the crystal in real space. Translating by a1 or a2 maps the pattern onto itself. Right: the same crystal in wavevector space (section 2.4).*

The lattice vectors are **a1 = a(1, 0)** and **a2 = a(1/2, √3/2)**, both of length a = 242.5 µm, 60° apart. The unit cell is the rhombus they span. Hole A sits at one third of the long diagonal, hole B at two thirds, so every A has three B neighbours at a distance a/√3 = 140 µm — a honeycomb, like graphene.

## 2.4 The wavevector k and the Brillouin zone

A wave travelling in some direction is described by its **wavevector k**: it points in the direction of travel and its length is |k| = 2π/λ (how fast the phase turns per µm). In a crystal, k and k + G behave identically when G is a *reciprocal lattice vector* (part 6.1), so all distinct waves are found inside one hexagon in k-space: the **Brillouin zone**. Special points get names: **Γ** (k = 0, the centre), **K** (a corner) and **M** (the middle of an edge). Because of the crystal's symmetry, the small orange triangle Γ–M–K already contains every distinct case.

Your COMSOL model walks along the triangle's edge with one number, also called k: **k = 0 at Γ, 1 at M, 2 at K, 3 back at Γ**. So 'k = 1 and 2' in your data simply means 'the points M and K'.

## 2.5 The band diagram, f_lower and f_upper

For every wavevector k the crystal allows only certain frequencies, the **bands**. Plotting them along Γ → M → K → Γ gives the **band diagram** (figure in part 6, exercise 5). Band 1 is the lowest, band 2 the next. If the highest point of band 1 is below the lowest point of band 2 there is a **complete band gap**:

$$f_{\text{lower}} = \max_{\mathbf{k}} f_1(\mathbf{k}), \qquad f_{\text{upper}} = \min_{\mathbf{k}} f_2(\mathbf{k}), \qquad \Delta f = f_{\text{upper}} - f_{\text{lower}}$$

In this project band 1 always peaks at K and band 2 bottoms out at M (or K), so two simulations per design are enough — the 'M/K shortcut', checked in section 6.4 of the option-B guide. If Δf < 0 the bands overlap and there is no complete gap.

## 2.6 What makes it a valley crystal

With two *identical* holes the pattern has six-fold symmetry and bands 1 and 2 touch at K (a **Dirac point**). Making the holes different breaks the symmetry between A and B; the touching point splits and a gap opens *at K*. The two corner types K and K′ ('valleys') then carry opposite topological charge, which is what protects the boundary waves. The bigger the difference between A and B, the wider the gap. Keeping three-fold symmetry is essential, which is why every hole shape in option B is three-fold symmetric (part 4).

**Exercise 1 — ex01_waves.py**

**Run:**

```
python learn\ex01_waves.py
```

**What it shows:** wavelength in air and silicon at 300 GHz and the normalised frequency a/λ.

**Then try:** F_GHZ = 252.72 and 321.84 (the IEEE 802.15.3d band edges); N_SI = 3.42.

### Check yourself

1. What is the wavelength in air at 250 GHz?
2. Why does a periodic structure have band gaps?
3. What do k = 1 and k = 2 mean in your COMSOL files?
4. Define $f_{lower}$ and $f_{upper}$.
5. If you make the whole crystal 10% larger, what happens to the gap?
6. Why do the two holes have to be different?

# 3. Fourier series from the beginning

## 3.1 The idea

Any pattern that repeats — a musical note, a heartbeat, the silicon/air pattern of a crystal — can be built by adding up simple waves (cosines and sines) whose frequencies are 1, 2, 3, ... times the repetition frequency. A chord is a sum of pure notes; a Fourier series is a recipe telling you *how much* of each pure note to add. The 'how much' numbers are the **Fourier coefficients**.

## 3.2 The formula (real form)

For a pattern g(x) that repeats every a:

$$g(x) = a_0 + \sum_{m=1}^{\infty} [a_m \cos(2\pi m x/a) + b_m \sin(2\pi m x/a)]$$

$$a_0 = \tfrac{1}{a}\int_0^a g\,dx, \qquad a_m = \tfrac{2}{a}\int_0^a g(x)\cos(2\pi m x/a)\,dx, \qquad b_m = \tfrac{2}{a}\int_0^a g(x)\sin(2\pi m x/a)\,dx$$

In words: **$a_0$ is the average** of the pattern; $a_m$ measures how much the pattern looks like the m-th cosine; $b_m$ the same for the m-th sine.

## 3.3 Why the formula works (orthogonality)

Multiply two cosines and average over one period. If they have *different* m the result is 0; if they have the same m it is 1/2. Example: the average of $\cos^2(2\pi x)$ over a period is 1/2 (because $\cos^2 = (1 + \cos 2\cdot 2\pi x)/2$ and the cosine part averages to 0), while the average of $\cos(2\pi x)\cdot\cos(4\pi x)$ is 0. So when you multiply g(x) by cos(2πmx/a) and average, every term of the series vanishes except the m-th, which leaves $a_m/2$. That is where the factor 2 in the formula comes from.

## 3.4 Worked example: the permittivity of a layered crystal

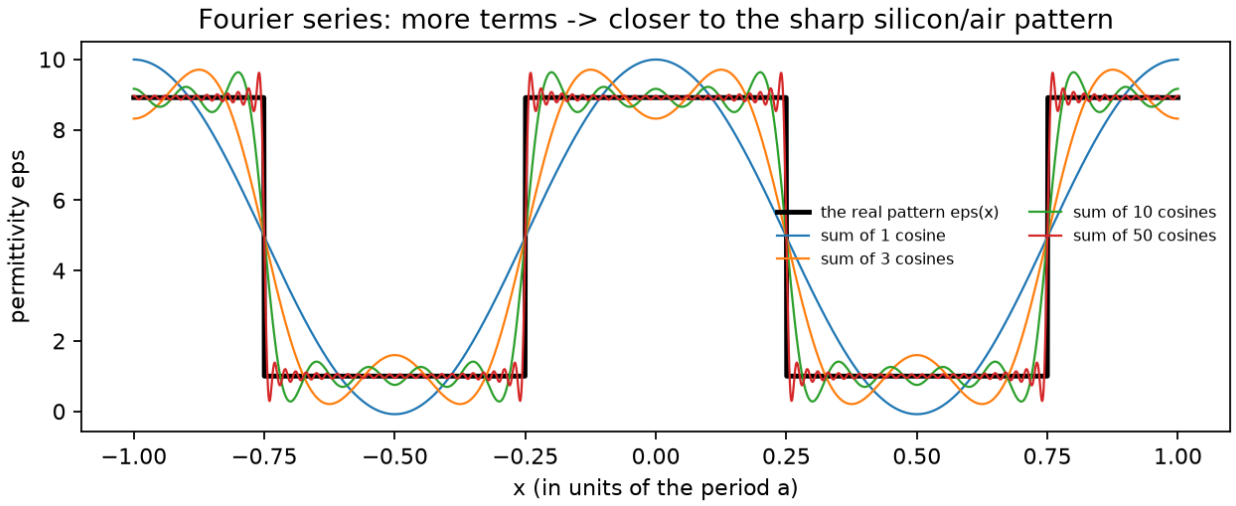
The 1D crystal of part 5: period a = 1, silicon (ε = 8.914) for −F/2 < x < F/2, air (ε = 1) elsewhere, with silicon fraction F = 0.5. Do this with pen and paper:

1. **Average.** $a_0$ = (silicon fraction)·$\varepsilon_{Si}$ + (air fraction)·1 = 0.5·8.914 + 0.5·1 = **4.957**.
2. **Split the integral.** Write ε(x) = 1 + ($\varepsilon_{Si}$ − 1)·[1 inside silicon, 0 outside]. The constant 1 gives zero for every m ≥ 1 (a cosine averages to zero over a period). So only the silicon part remains:

$$a_m = 2(\varepsilon_{\mathrm{Si}} - 1)\int_{-F/2}^{F/2} \cos(2\pi m x)\,dx = 2(\varepsilon_{\mathrm{Si}} - 1)\,\frac{\sin(\pi m F)}{\pi m}$$

3. **m = 1:** $a_1$ = 2·7.914·sin(π/2)/π = 15.828/3.1416 = **5.038**.
4. **m = 2:** sin(π) = 0, so $a_2$ = **0** (with F = 0.5 all even terms vanish).
5. **m = 3:** $a_3$ = 15.828·sin(3π/2)/(3π) = −15.828/9.425 = **−1.679**.
6. **Sines:** the pattern is symmetric about x = 0 (an even function) and sines are odd, so every $b_m$ = 0.

Exercise 2 computes the same numbers with the computer (FFT) and prints them next to the formula: 4.957, 5.038, 0, −1.679, 0, 1.008 — identical.




*Exercise 2: adding more and more cosines reproduces the sharp silicon/air pattern. Near the jumps a small overshoot remains (the Gibbs effect); smooth shapes need far fewer terms.*

## 3.5 The complex form (used by the PWE solver)

Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$ lets us write cosine and sine together. The series becomes a sum over positive *and negative* m:

$$g(x) = \sum_{m=-\infty}^{\infty} c_m\, e^{\,i2\pi mx/a}, \qquad c_m = \tfrac{1}{a}\int_0^a g(x)\, e^{\,-i2\pi mx/a}\, dx$$

For a symmetric (even) real pattern: $c_0 = a_0$ and $c_m = c_{-m} = a_m/2$. Example: $c_1 = 5.038/2 = 2.519$ — you will meet this exact number inside the PWE matrix in part 5. Why bother with complex numbers? Because derivatives become multiplications: $d/dx\, e^{iqx} = iq\cdot e^{iqx}$. That turns a differential equation into an ordinary matrix problem.

## 3.6 The FFT

The **Fast Fourier Transform** is the computer's way of getting all $c_m$ at once from N equally spaced samples of the pattern: in code, `np.fft.fft(samples) / N`. The 2D version (`np.fft.fft2`) does the same for a 2D picture of the unit cell — that is step 4 of the project's solver.

**Exercise 2 — ex02_fourier_series.py**

**Run:**

```
python learn\ex02_fourier_series.py
```

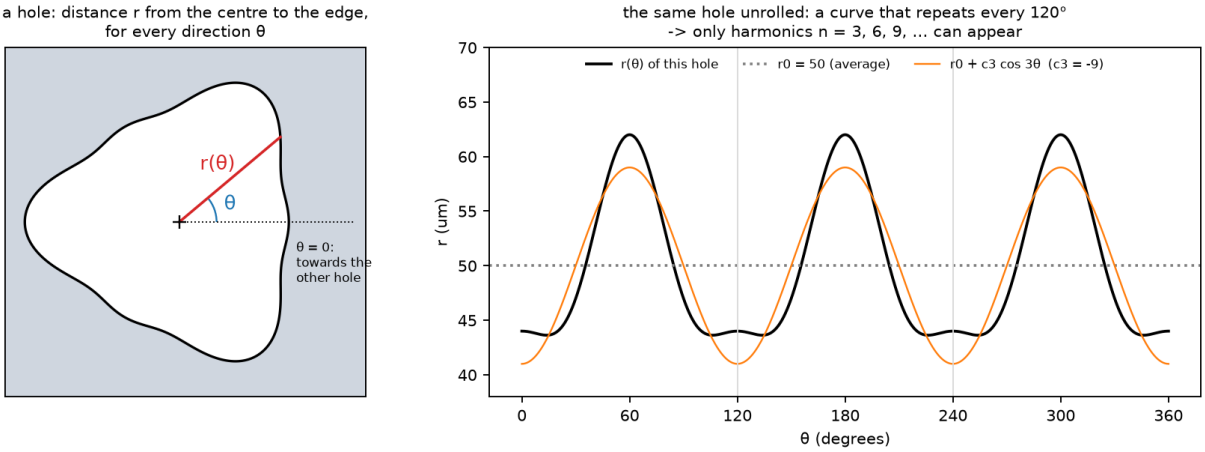
**What it shows:** the coefficients by formula and by FFT; the pattern rebuilt from 1, 3, 10 and 50 cosines.
**Then try:** FILL = 0.2 and 0.8 (which coefficients vanish now?); N_TERMS = (1, 2, 5, 200).

### Check yourself

1. What does $a_0$ represent?
2. Why are all $b_m$ zero for the layer pattern centred at x = 0?
3. Compute $a_1$ for F = 0.25 (ε_Si = 8.914).
4. How are $c_1$ and $a_1$ related for an even pattern?
5. Why do sharp edges need many Fourier terms?

# 4. Describing a hole shape with a Fourier series

## 4.1 A hole as a function r(θ)




*Left: from the hole centre, look in direction ϑ and measure the distance r to the edge. Right: the same numbers plotted against ϑ — a curve that repeats every 360° (here even every 120°).*

Stand at the centre of a hole and look in direction θ. The distance to the edge is r(θ). Going once around (θ from 0 to 360°) you come back to the start, so **r(θ) is a repeating pattern with period 360°** — exactly what part 3 says can be written as a Fourier series, now in the angle θ instead of position x:

$$r(\theta) = r_0 + \sum_n [c_n \cos n\theta + s_n \sin n\theta]$$

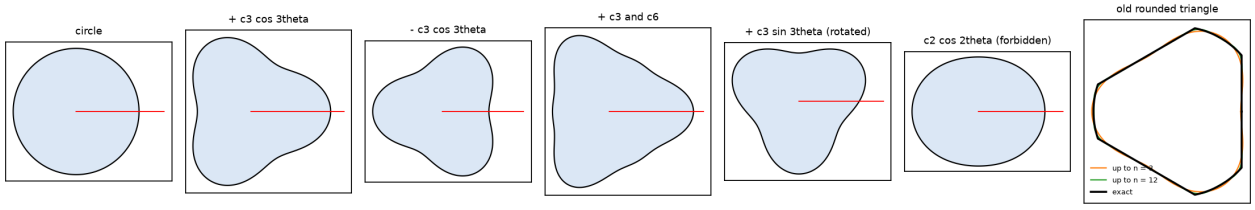
$r_0$ is the average radius (the size), $c_n$ and $s_n$ describe the shape. A **circle** is just r(θ) = $r_0$: every other coefficient is zero.

## 4.2 Worked example: adding c3 cos 3θ

Take $r_0$ = 50 µm and $c_3$ = 10 µm. Fill in the table yourself, then compare with exercise 3:

| θ | 3θ | cos 3θ | r = 50 + 10 cos 3θ (µm) |
|---|---|---|---|
| 0° | 0° | 1 | 60 (a bump) |
| 30° | 90° | 0 | 50 |
| 60° | 180° | −1 | 40 (a dent) |
| 90° | 270° | 0 | 50 |
| 120° | 360° | 1 | 60 (the next bump) |
| 180° | 540° | −1 | 40 |

Three bumps (0°, 120°, 240°) and three dents: a **rounded triangle**. Negative $c_3$ swaps bumps and dents (the triangle points the other way). $s_3$ sin 3θ is the same shape turned by 30°. Adding $c_6$ cos 6θ sharpens or flattens the corners; $c_9$, $c_{12}$ add finer detail.




*Exercise 3: circle; ± c3; c3 with c6; the sine version; the forbidden cos 2ϑ term (two-fold, not three-fold); and the old rounded triangle rebuilt from its first Fourier terms. Red line = ϑ = 0, the direction towards the neighbouring hole.*

## 4.3 The three-fold rule, step by step

Valley crystals need each hole to look the same after a turn of 120°: r(θ + 120°) = r(θ) for every θ. Test one term:

$$\cos\left(n(\theta + 120^\circ)\right) = \cos\left(n\theta + n \cdot 120^\circ\right)$$

This equals cos nθ for every θ only if the extra angle n·120° is a whole number of turns (a multiple of 360°):

| n | n·120° | whole turns? | allowed? |
|---|---|---|---|
| 1 | 120° | no | no |
| 2 | 240° | no | no |
| 3 | 360° | 1 | yes |
| 4 | 480° | no | no |
| 6 | 720° | 2 | yes |
| 9 | 1080° | 3 | yes |
| 12 | 1440° | 4 | yes |

So **only n = 3, 6, 9, 12, ...** may appear. Exercise 3 confirms it numerically: the shapes built from n = 3 and 6 change by 0.000 µm when turned by 120°, the cos 2θ shape by 8.66 µm. This is a **symmetry selection rule**: the physics forbids most coefficients, so the description is short.

## 4.4 Nine numbers per hole, eighteen per cell

The project keeps n = 3, 6, 9, 12, each with a cosine and a sine, plus $r_0$: **1 + 2·4 = 9 numbers per hole**, in the order [r0, c3, s3, c6, s6, c9, s9, c12, s12]. Two holes give **18 numbers**: the input of every machine-learning model. The same 18 numbers describe a circle, a sharp triangle, a rounded triangle or any freeform shape — that is why one model can learn from all datasets together.

Each hole measures θ from its own direction 'towards a neighbour of the other type' (θ = 0). For hole A that direction is 30° in the lab; for hole B it is 210° — the two face each other.

## 4.5 Worked example: the old rounded triangle

The old COMSOL holes are triangles of side L cut by a circle of radius 2L/(3√3) (option-B guide, 3.3). A flat side faces θ = 0, so r is *smallest* at θ = 0 and largest at the corners (60°, 180°, 300°). Exercise 3 evaluates r(θ) at 3,600 angles for L = 168 µm and applies the formulas of part 3:

| n | 0 | 1, 2 | 3 | 4, 5 | 6 | 7, 8 | 9 | 10, 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|
| c_n (µm) | 56.87 | 0 | −9.08 | 0 | 0.10 | 0 | 1.11 | 0 | −0.63 |

Everything not a multiple of 3 is zero, as the rule says. $c_3$ is negative because the flat side (small r) is at θ = 0. Dividing: $r_0/L = 0.339$ and $c_3/r_0 = -0.160$ — any rounded triangle of any size has these ratios.

## 4.6 The design space and the fabrication rules

For inverse design the project uses **10 numbers**: [r0, c3, s3, c6, s6] for each hole (c9 to s12 set to zero to keep shapes smooth). Allowed ranges: r0 from 25 to 75 µm, |c3|, |s3| up to 0.35·r0, |c6|, |s6| up to 0.12·r0. The optimisers work with u in $(0, 1)^{10}$ and convert, e.g. r0 = 25 + 50u, so every u inside the box is a legal shape.

A shape must also be **fabricable**: silicon walls between holes at least 25 µm, radius never below 10 µm, corners not sharper than an 8 µm radius, holes not overlapping. `geometry.check_cell` tests this. Exercise 6 shows a design with r0 = 95 µm failing: its thinnest wall is 10 µm.

**Exercise 3 — ex03_hole_shapes.py**

**Run:**

```
python learn\ex03_hole_shapes.py
```

**What it shows:** the table of 4.2, the 120° symmetry test, and the Fourier numbers of the old rounded triangle.
**Then try:** C3 = −0.2·R0; C2 = 0.3·R0 (look at the symmetry test); L_TRI = 140.

### Check yourself

1. Which Fourier terms can a three-fold symmetric hole have, and why?
2. What shape is r = 40 + 0·cos 3θ?
3. How do you rotate a c3 cos 3θ shape by 30°?
4. Why is c3 of the old rounded triangle negative?
5. How many input numbers do the models get, and why that many?

# 5. The plane-wave expansion (PWE) method in 1D

This is the heart of the project's fast simulator. We first do it for a 1D crystal (layers), small enough to write out by hand; part 6 only adds a second dimension.

## 5.1 Eigenvalue problems in one paragraph

For a square matrix A, an **eigenvector** v is a direction that A only stretches: A v = λ v; the stretch factor λ is the **eigenvalue**. Example: A = [[2, 1], [1, 2]]. Try v = (1, 1): A v = (3, 3) = 3·v, so λ = 3. Try v = (1, −1): A v = (1, −1), so λ = 1. A computer finds all of them with `np.linalg.eigvalsh(A)`. In photonics the 'stretch factors' are the allowed frequencies (squared), and the eigenvectors are the shapes of the waves.

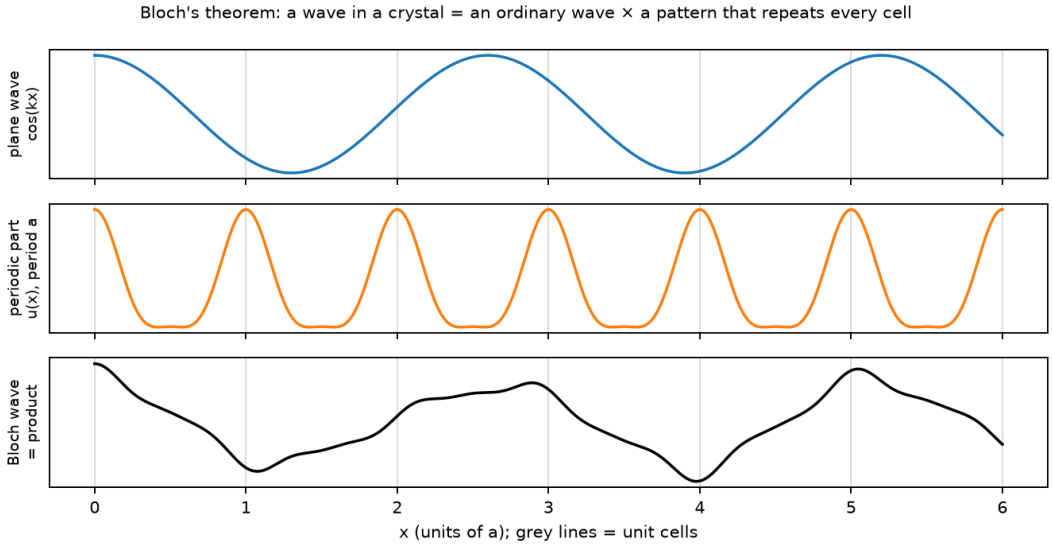
## 5.2 The wave equation in the crystal

For a wave crossing the layers, Maxwell's equations reduce to one equation for the magnetic field H(x):

$$-\frac{d}{dx}\left[\frac{1}{\varepsilon(x)}\frac{dH}{dx}\right] = \left(\frac{\omega}{c}\right)^2 H(x), \qquad \omega = 2\pi f$$

Read it as: 'an operation on H gives back H times a number'. That is an eigenvalue problem: the allowed $(\omega/c)^2$ are the eigenvalues. The only thing describing the crystal is ε(x).

## 5.3 Bloch's theorem: what a wave in a crystal looks like




*A Bloch wave: an ordinary wave $e^{ikx}$ multiplied by a pattern u(x) that repeats every unit cell.*

Because the crystal repeats, every allowed wave is a plane wave times a periodic function: $H(x) = e^{ikx}\cdot u(x)$, with u(x + a) = u(x). And a periodic function has a Fourier series (part 3)! So, with $G_m = 2\pi m/a$:

$$H(x) = e^{ikx}\sum_m h_m e^{iG_m x} = \sum_m h_m\, e^{i(k+G_m)x}$$

The field is a sum of **plane waves** with wavevectors $k + G_m$ — hence the name 'plane-wave expansion'. The unknowns are the amplitudes $h_m$.

## 5.4 Turning the equation into a matrix

Also expand $1/\varepsilon(x) = \Sigma\, \eta_m\, e^{iG_m x}$. Now substitute, one step at a time:

1. dH/dx: each term $e^{i(k+G_{m'})x}$ becomes $i(k+G_{m'})\cdot e^{i(k+G_{m'})x}$.
2. Multiply by $1/\varepsilon$: $\eta_j e^{iG_j x}$ times $e^{i(k+G_{m'})x}$ gives $e^{i(k+G_{j+m'})x}$, because exponents add ($G_j + G_{m'} = G_{j+m'}$).
3. Differentiate again: another factor $i(k+G_m)$ for the term that now has index m = j + m'.

4. Collect all terms in front of the same plane wave $e^{i(k+G_m)x}$ (that is j = m − m') and compare both sides. The two factors of i give i·i = −1, which cancels the minus sign of the equation:

$$\sum_{m'}(k+G_m)\ \eta_{m-m'}\ (k+G_{m'})\ h_{m'} = (\tfrac{\omega}{c})^2 h_m$$

That is **matrix × vector = number × vector**: the matrix entries are $M_{mm'} = (k+G_m)\ \eta_{m-m'}\ (k+G_{m'})$. Keep the plane waves m = −M … M and you have a (2M+1) × (2M+1) eigenvalue problem. Solve it at one k: the eigenvalues give the frequencies of all bands at that k. Repeat for many k: you have the band diagram.

## 5.5 Ho's rule: invert the ε matrix

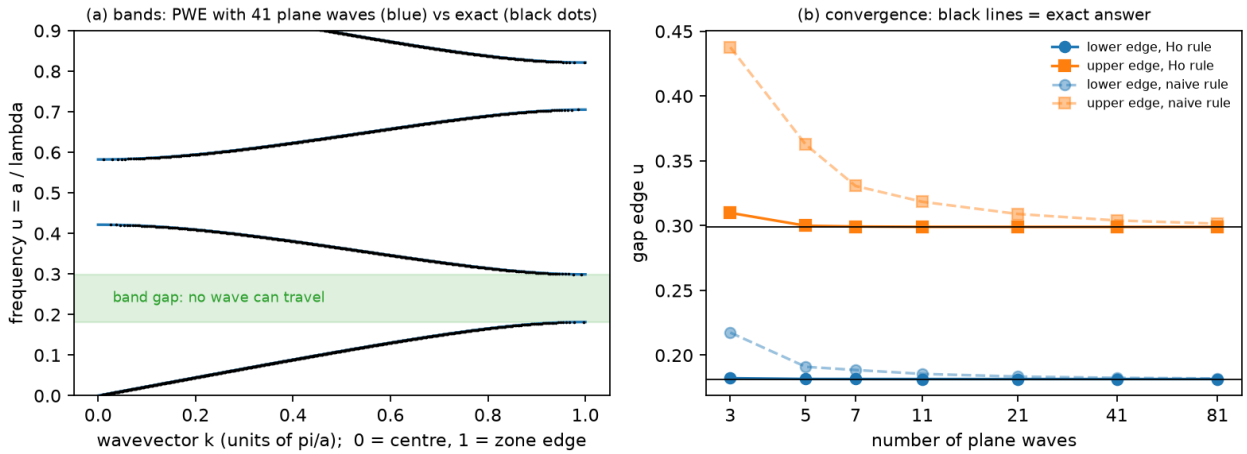
There are two ways to get the η matrix. The naive way: Fourier coefficients of 1/ε directly. **Ho's rule** (Ho, Chan and Soukoulis 1990): build the matrix of ε coefficients $[c_{m-m'}]$ (a *Toeplitz* matrix — constant along each diagonal) and **invert** it. Both are exact with infinitely many plane waves, but with a finite number Ho's rule handles the sharp silicon/air jumps far better — see 5.7. The project's solver uses Ho's rule.

## 5.6 Worked example with only 3 plane waves

Silicon fraction 0.5, ε = 8.914, units a = 1, wave at the zone edge k = π. Plane waves m = −1, 0, 1 so k + G = π − 2π, π, π + 2π = **−π, π, 3π**.

1. Coefficients of ε (part 3.5): $c_0 = 4.957$, $c_{\pm1} = 2.519$, $c_{\pm2} = 0$.
2. Toeplitz matrix $[c_{m-m'}]$ (row m, column m'): [[4.957, 2.519, 0], [2.519, 4.957, 2.519], [0, 2.519, 4.957]].
3. Invert it (computer, or by hand with patience) to get η.
4. Multiply entry (m, m') by $(k+G_m)(k+G_{m'})$. Exercise 4 prints the result: [[3.055, 2.093, −3.191], [2.093, 4.118, −6.278], [−3.191, −6.278, 27.491]].
5. Eigenvalues: 1.311, 3.791, 29.56. These are $(\omega/c)^2$, so ω/c = 1.145, 1.947, ...; the normalised frequency u = a/λ = (ω/c)/(2π) = **0.182** and **0.310**.

The exact answer for this crystal (a textbook formula, also in exercise 4) has its first gap between u = 0.1816 and 0.2990. With just **3 plane waves** the lower edge is already right to 0.3% and the upper to 4%. For the real a = 242.5 µm multiply u by c/a = 1236 GHz: the gap is 224.5 – 369.7 GHz.




*Exercise 4: (a) the bands for all k with 41 plane waves (blue) lie exactly on the exact solution (black dots); green = the band gap. (b) Gap edges vs the number of plane waves: Ho's rule (solid) converges almost at once, the naive rule (dashed) slowly.*

## 5.7 Convergence: how many plane waves?

| plane waves | Ho: lower | Ho: upper | naive: lower | naive: upper |
|---|---|---|---|---|
| 3 | 0.1822 | 0.3099 | 0.2175 | 0.4377 |
| 5 | 0.1816 | 0.2999 | 0.1910 | 0.3625 |
| 11 | 0.1816 | 0.2991 | 0.1856 | 0.3184 |

| plane waves | Ho: lower | Ho: upper | naive: lower | naive: upper |
|---|---|---|---|---|
| 41 | 0.1816 | 0.2990 | 0.1826 | 0.3040 |
| 81 | 0.1816 | 0.2990 | 0.1821 | 0.3015 |
| exact | 0.1816 | 0.2990 | 0.1816 | 0.2990 |

Lesson: more plane waves = more accuracy but a bigger matrix (cost grows like the cube of its size). Ho's rule gives the accuracy cheaply, which is what makes 0.2 seconds per design possible in 2D.

**Exercise 4 — ex04_pwe_1d.py**

**Run:**

```
python learn\ex04_pwe_1d.py
```

**What it shows:** the 3 × 3 matrices of 5.6, the convergence table and the band diagram.

**Then try:** FILL = 0.3; EPS_SI = 11.7 (does the gap get wider?); M_PLOT = 2.

### Check yourself

1. What does Bloch's theorem say?
2. Why does a derivative become a multiplication in PWE?
3. Where do the matrix's off-diagonal entries come from?
4. What do the eigenvalues of the PWE matrix mean?
5. What is Ho's rule and why use it?
6. With u = 0.25 and a = 242.5 µm, what is the frequency?

# 6. The 2D solver of the project (src/shapes/pwe.py)

## 6.1 Reciprocal lattice vectors

In 2D the periodic part of the field repeats along a1 and a2, so its Fourier series uses vectors G = m·b1 + n·b2 with two whole numbers m, n. The b's are chosen so that $b_i \cdot a_j = 2\pi$ if i = j and 0 otherwise:

$$\mathbf{b}_1 = \frac{2\pi}{a}\left(1,\ -\frac{1}{\sqrt{3}}\right), \qquad \mathbf{b}_2 = \frac{2\pi}{a}\left(0,\ \frac{2}{\sqrt{3}}\right)$$

**Check one by hand:** a1·b1 = a(1, 0)·(2π/a)(1, −1/√3) = 2π (correct). a2·b1 = a(1/2, √3/2)·(2π/a)(1, −1/√3) = 2π(1/2 − 1/2) = 0 (correct). The Brillouin zone of part 2.4 is the hexagon around the origin of this G-lattice; M = (b1 + b2)/2 and K = (4π/3a, 0).

## 6.2 The 2D equation

For the polarisation used here (TE: magnetic field $H_z$ out of the plane, electric field in the plane) the 1D equation keeps its form; the products of wavenumbers become dot products of vectors:

$$\sum_{\mathbf{G}'} (\mathbf{k}+\mathbf{G})\cdot(\mathbf{k}+\mathbf{G}')\,[\varepsilon^{-1}]_{\mathbf{G}\mathbf{G}'}\,h_{\mathbf{G}'} = \left(\frac{\omega}{c}\right)^2 h_{\mathbf{G}}$$

## 6.3 What the code does, step by step

1. **Grid.** The unit cell is divided into 96 × 96 pixels, each further split into 4 × 4 sub-pixels.
2. **Holes.** For every sub-pixel the code knows its distance ρ and angle θ from the nearest hole A and B (computed once). A sub-pixel is air if ρ < r(θ) — *this is where the Fourier hole shape of part 4 enters*. Any family works: circle, triangle or the 9 Fourier numbers.
3. **Smooth ε.** Each pixel's ε is the average of its 16 sub-pixels (edge pixels get in-between values), so a tiny change of shape gives a tiny change of frequency — important for gradient-based design.
4. **FFT.** `np.fft.fft2` of the ε picture gives the coefficients $\varepsilon_G$.
5. **Toeplitz and Ho's rule.** Keep the 439 vectors G with |G| ≤ 11|b1|. Build the 439 × 439 matrix $[\varepsilon_{G-G'}]$ and invert it — once per design.
6. **Per k-point.** Multiply entry (G, G') by (k+G)·(k+G'), find the 2 smallest eigenvalues (`scipy.linalg.eigh`), convert: f = c·√λ / (2π).
7. **Band edges.** Do step 6 at M and at K only: $f_{lower}$ = max(band 1 at M, at K), $f_{upper}$ = min(band 2 at M, at K).

The core of steps 4–6 in the real code (simplified):

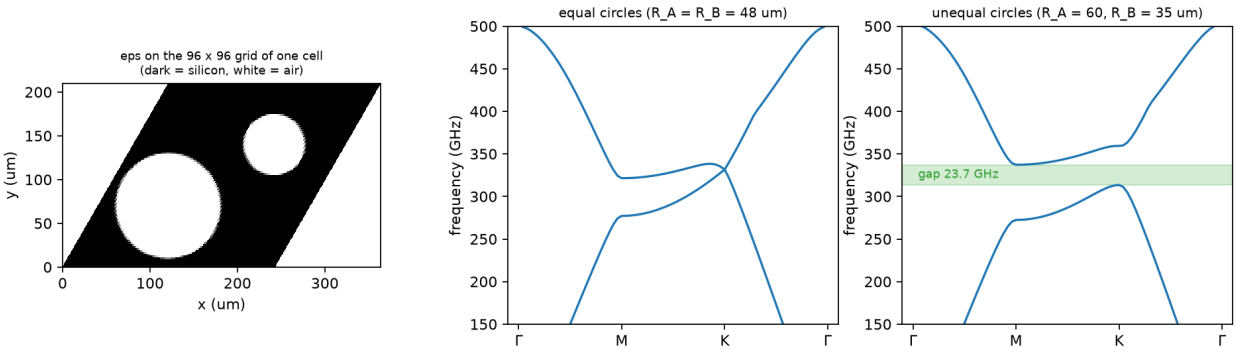
```
eps = self.eps_grid(holeA, holeB)             # steps 1-3: 96 x 96 picture of eps
eg = np.fft.fft2(eps) / eps.size               # step 4: Fourier coefficients
T = eg[self.dm, self.dn]                       # step 5: Toeplitz matrix eps_(G-G')
inv = np.linalg.inv(T)                         #         Ho's rule
kv = kvec + self.G                             # step 6: all vectors k + G
M = (kv @ kv.T) * inv                          #         (k+G).(k+G') [eps^-1]
lam = eigh(M, eigvals_only=True, subset_by_index=[0, 1])   # two lowest bands
f_GHz = np.sqrt(lam) * 1e6 * C0 / (2 * np.pi) / 1e9          # lengths are in um
```

## 6.4 Accuracy and speed

Exercise 5 repeats the convergence test of part 5 in 2D (circles of 60 and 35 µm). From 91 to 693 plane waves the gap changes from 22.95 to 23.82 GHz; the project uses 439 (23.72 GHz), within about 0.1–0.4 GHz of the converged value. Setting up the solver takes 0.17 s, one design at M and K about 0.2 s. Against COMSOL the solver is within about 1 GHz for all 177 COMSOL cells (option-B guide, 6.3).

## 6.5 Calibration: why ε = 8.91 and not 11.7

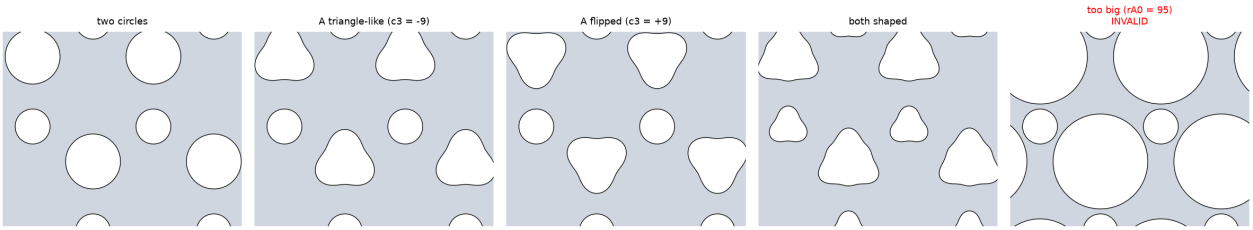
With bulk silicon's ε = 11.7 every PWE frequency came out about 13% too low compared with COMSOL — the same ratio for every design and every k. A constant ratio means a different material constant, not a wrong geometry. Fitting ε gives 8.914 (n = 2.986), after which all triangle cells agree within about 1 GHz, and the circle cells (never used in the fit) agree just as well. The likely reason: the 2D COMSOL model uses an *effective index* for the thin slab; Shalaev et al. (2019) used $n_{eff}$ = 2.965 the same way. Please confirm in the COMSOL material settings.




*Exercise 5: left, the ε picture the solver builds (dark = silicon). Middle, equal circles: bands 1 and 2 touch at K (Dirac point, 331.6 GHz), and band 2 at M (321.5 GHz) dips below it — no complete gap. Right, unequal circles: a 23.7 GHz valley gap opens.*

## 6.6 From design numbers to band edges

Exercise 6 runs the complete forward chain that the inverse methods must invert: 10 design numbers → two Fourier holes → fabrication check → PWE band edges. Holes with the *same average radii* (55 and 35 µm) but different shapes give gaps from 10.4 to 15.4 GHz — the shape matters, which is the whole point of option B.




*Exercise 6: five designs. The last one breaks the fabrication rules (10 µm wall) and is never simulated.*

**Exercise 5 — ex05_pwe_2d.py and ex06_design_to_band_edges.py**

**Run:**

```
python learn\ex05_pwe_2d.py
python learn\ex06_design_to_band_edges.py
```

**What it shows:** the solver's ε picture, band diagrams for equal and unequal holes, convergence, and the full forward chain.

**Then try:** R_A, R_B = 55, 40 in ex05; add your own design to DESIGNS in ex06 and predict the gap before running.

### Check yourself

1. What is a reciprocal lattice vector?
2. Where does the hole shape enter the PWE solver?
3. Why average 16 sub-pixels per pixel?
4. What is computed once per design and what once per k-point?
5. Why was ε fitted?

# 7. Machine learning: learning a function from examples

## 7.1 What a model and training are

A **model** is a formula with adjustable numbers called **parameters**. **Training** = choosing the parameters so the model's outputs match known examples. The mismatch is measured by a **loss**, usually the mean squared error (MSE) = average of (prediction − truth)².

**Worked example: gradient descent by hand.** Model y = w·x, one parameter w. Examples: (x, y) = (1, 2) and (2, 4) — the right answer is w = 2. Loss $L(w) = \frac{1}{2}[(w\cdot 1 - 2)^2 + (w\cdot 2 - 4)^2]$. Its slope (derivative) is $dL/dw = (w - 2)\cdot 1 + (2w - 4)\cdot 2 = 5w - 10$. **Gradient descent**: repeatedly step downhill, w ← w − (learning rate)·dL/dw. Start at w = 0 with learning rate 0.1:

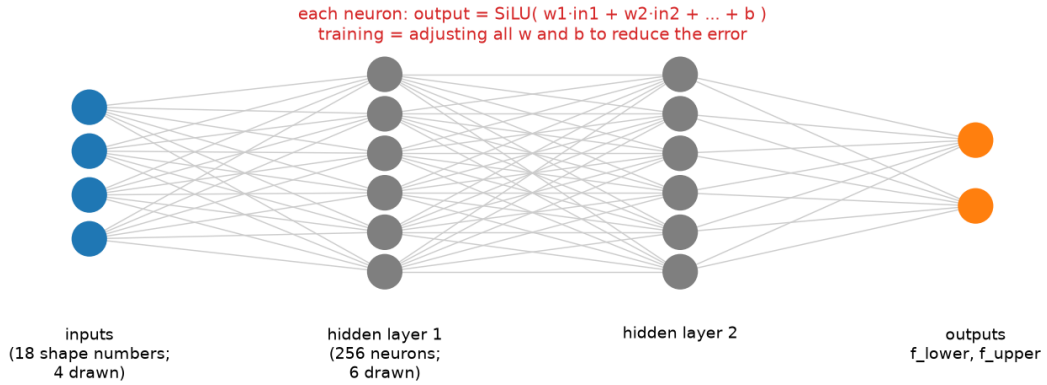
| step | w | slope 5w − 10 | new w = w − 0.1·slope |
|---|---|---|---|
| 1 | 0 | −10 | 1.0 |
| 2 | 1.0 | −5 | 1.5 |
| 3 | 1.5 | −2.5 | 1.75 |
| 4 | 1.75 | −1.25 | 1.875 |

w approaches 2. A neural network does exactly this, with about 137,000 parameters instead of one; the computer finds all the slopes at once with *backpropagation* (PyTorch's `loss.backward()`). **Adam** is an improved gradient descent that adapts the step size of every parameter.

## 7.2 Training, validation, test — and honest errors

If you judge a model on the examples it learned from, it looks perfect even when it has only memorised them (**overfitting**). So the data are split: **training** (to fit), **validation** (to choose settings) and **test** (touched once, at the end). The project reports the **mean absolute error (MAE) in GHz** on test cells — a number you can compare with the 1 GHz agreement between PWE and COMSOL. Repeating with different random splits (the 'seeds' you ran overnight) shows how much a result depends on luck.

## 7.3 Neural networks




*A multi-layer perceptron (MLP). Every line is a weight; every neuron adds up its weighted inputs and applies a bend (the activation).*

Each neuron computes output = SiLU($w_1 \cdot in_1 + w_2 \cdot in_2 + \ldots + b$). SiLU(z) = $z/(1 + e^{-z})$ is a smooth bend; without bends the whole network would be a straight-line formula. The project's forward network: 18 inputs → 3 hidden layers of 256 → 2 outputs. **Count the parameters:** 18·256 + 256 = 4,864; two layers of 256·256 + 256 = 65,792 each; 256·2 + 2 = 514; total **136,962**. Inputs and outputs are normalised (minus mean, divided by standard deviation) so all numbers are of similar size.

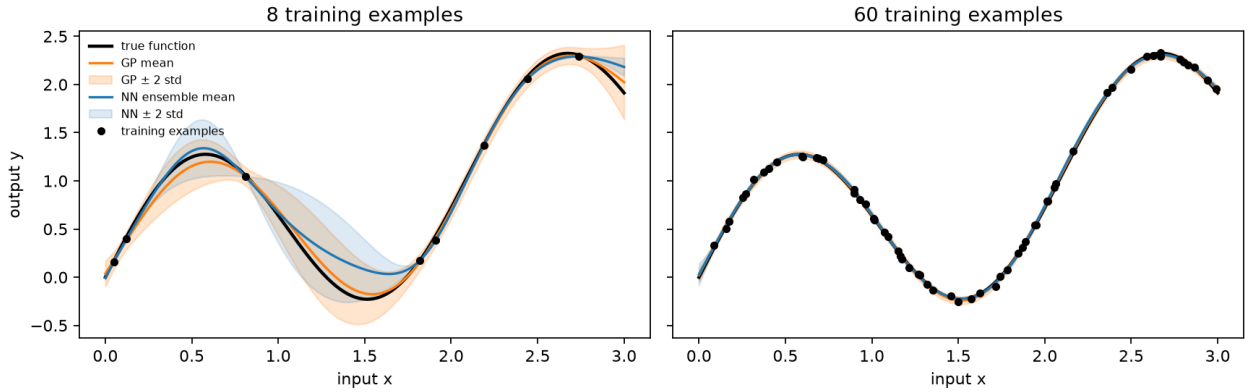
**Ensemble.** Train 5 networks from different random starts. Their average is the prediction; their disagreement is an uncertainty estimate — where they disagree, the data were thin.

## 7.4 Gaussian processes

A **Gaussian process (GP)** works on one assumption: *similar inputs give similar outputs*. 'Similar' is defined by a **kernel**, here the RBF kernel

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\sum_j \frac{(x_j - x'_j)^2}{2\ell_j^2}\right)$$

A prediction at a new x is a weighted combination of the training outputs, with large weights for nearby examples. Because it knows how far x is from the data, a GP also gives a **standard deviation**: small near examples, large far away. Each length scale $l_j$ says how far one must move along feature j before the output changes ('ARD'); a huge $l_j$ means feature j barely matters. The cost: training needs to invert an n × n matrix, so time grows like $n^3$ — fine for 100s of examples, slow for 1000s.




*Exercise 7: with 8 examples the GP (orange) is better than the network ensemble (blue) and its uncertainty band shows where it is guessing. With 60 examples both are accurate.*

Exercise 7 prints: with 8 examples GP error 0.045 vs network 0.080; with 60 examples 0.008 vs 0.010. The project's learning curve shows the same pattern at a larger scale: the GP wins with few hundred cells, the network keeps improving with thousands (option-B guide, chapter 8). This is also why, in stage 1 (only L1 and L2, 61 cells), a GP beat the neural networks.

**Exercise 7 — ex07_surrogate_gp_vs_nn.py**

**Run:**

```
python learn\ex07_surrogate_gp_vs_nn.py
```

**What it shows:** GP and network ensemble learning the same function from 8 and 60 examples, with uncertainty bands.

**Then try:** N_SMALL = 4; HIDDEN = 8; STEPS = 300 (under-trained).

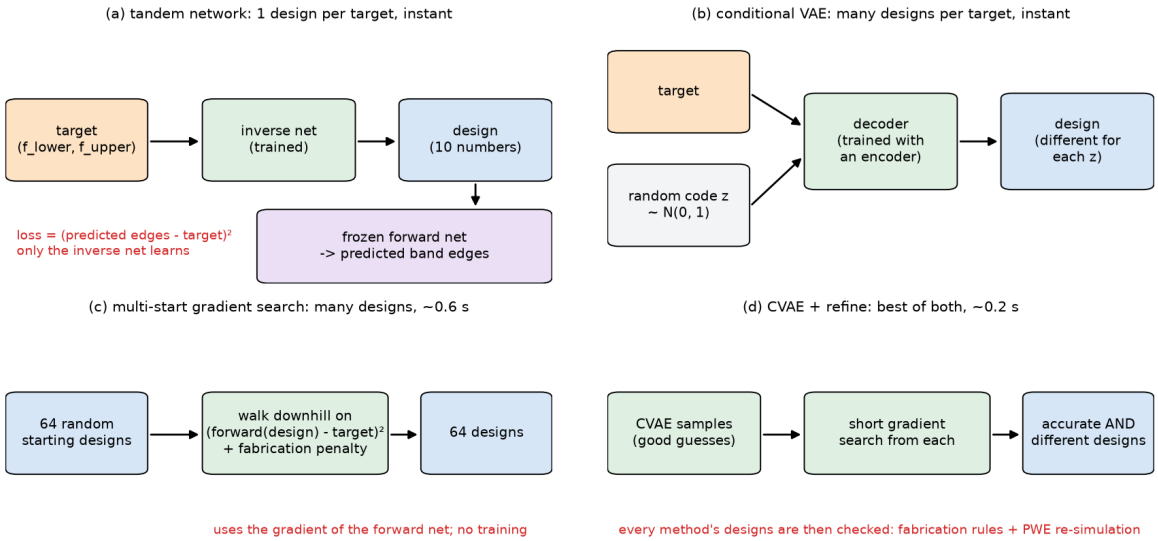
### Check yourself

1. What is a loss function?
2. Do one more gradient-descent step of the table in 7.1.
3. Why keep a test set aside?
4. What does the spread of an ensemble tell you?
5. When would you prefer a GP over a neural network?
6. Why does the project's network have SiLU between layers?

# 8. Inverse design: running the model backwards

## 8.1 The problem, and why it is hard

The forward surrogate (part 7) answers 'shape → band edges' instantly. Inverse design asks 'band edges → shape'. With 10 design numbers and only 2 targets, **many shapes give the same band edges** (the option-B gallery found 24 different verified shapes for one target). The toy used in exercises 8–10 has the same structure: design x = (x1, x2) in a square, response $y = x1^2 + x2^2$. For y = 0.5 every point on the circle of radius √0.5 is a correct design.




*The four inverse methods compared in the project. All of them use the forward surrogate; all their designs are re-checked with the fabrication rules and the PWE solver.*

## 8.2 Why the direct inverse fails

Obvious idea: train a network on examples (y → x). But for y = 0.5 it has seen (0.7, 0), (−0.7, 0), (0, 0.7), (0, −0.7) … and minimising the squared error makes it predict their **average**, near (0, 0) — whose response is y = 0, nowhere near 0.5. Exercise 8 shows exactly this: every direct design lands at the centre, average miss 0.52.

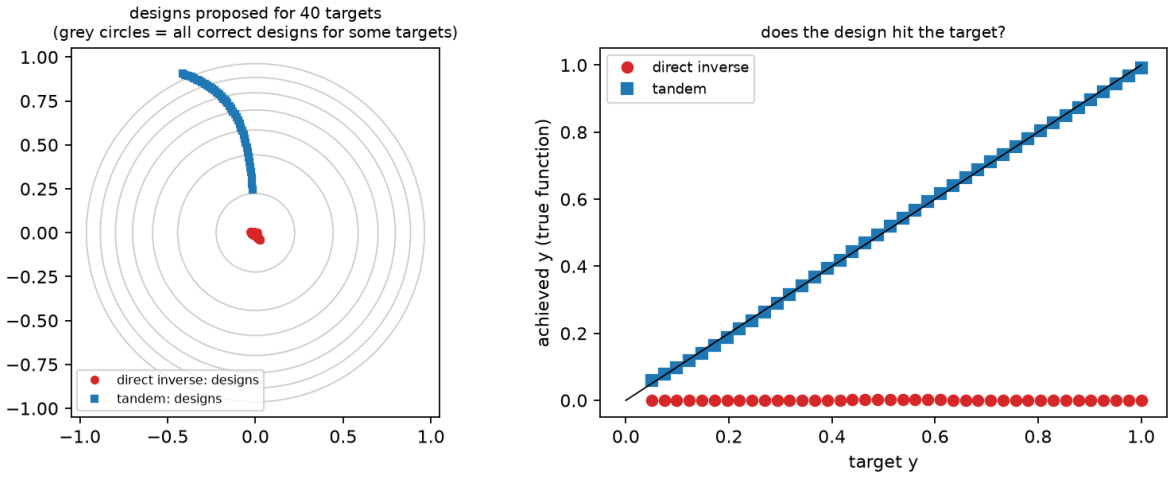
## 8.3 The tandem network (Liu et al. 2018)

Two steps. (1) Train the **forward** network x → y as usual. (2) **Freeze** it and put a new inverse network in front of it. Train only the inverse network, with the loss measured on the *response*:

$$\mathcal{L} = \left| F(G(\mathbf{t})) - \mathbf{t} \right|^2 + \lambda \, P(G(\mathbf{t}))$$

G is the inverse network, F the frozen forward network, t the target, P the fabrication penalty. The inverse network is no longer forced to reproduce one particular stored design, only *some* design that works, so it settles on one consistent family of answers. Exercise 8: average miss 0.005 instead of 0.52.

In the project: G outputs u in $(0, 1)^{10}$ through a sigmoid (always a legal shape), and the penalty pushes it away from thin walls. Strengths: instant (one network evaluation) and the most accurate method. Weakness: **one design per target** — no alternatives.




*Exercise 8: left, direct-inverse designs (red) collapse to the centre; tandem designs (blue) sit on the correct circles. Right: achieved vs target response.*

## 8.4 Conditional variational autoencoder (CVAE)

Goal: **many** designs per target. The trick is a short random code z (here 1 number, in the project 4) that selects *which* of the correct designs you get.

**Training** uses two networks: an **encoder** reads a known design and its response (x, y) and outputs a code z; a **decoder** reads (z, y) and must rebuild x. The loss has two parts:

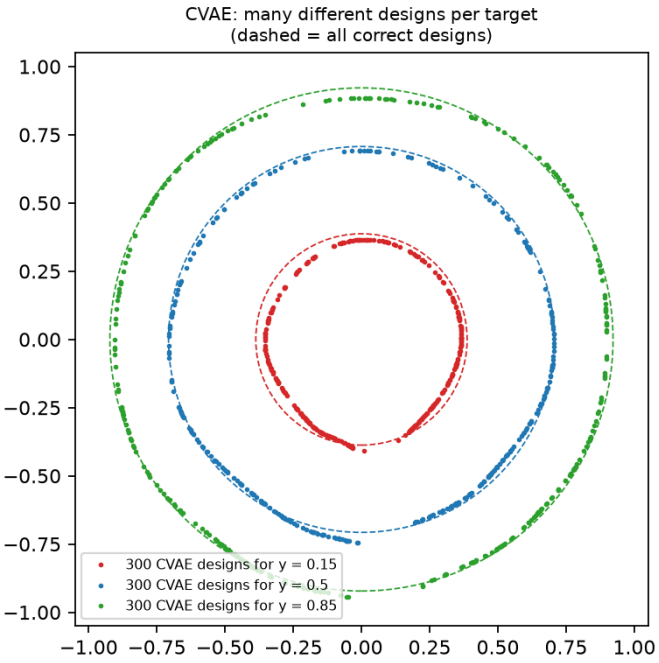
$$\mathcal{L} = |\hat{\mathbf{x}} - \mathbf{x}|^2 + \beta\,\mathrm{KL}(q(z|\mathbf{x}, y)\ \|\ \mathcal{N}(0, 1))$$

The first term says 'rebuild the design'; the second says 'make the codes look like a standard normal distribution N(0, 1)'.

The KL term (Kullback–Leibler divergence) pushes the codes to fill a standard normal distribution with no gaps, so that *any* random code is meaningful later. For a normal distribution with mean μ and variance σ² it is ½(μ² + σ² − 1 − ln σ²), zero exactly when μ = 0 and σ = 1. To train through the random sampling we write z = μ + σ·(random normal number) — the **reparameterisation trick**.

**Using it:** throw the encoder away. For a target y, draw 300 random codes z and decode each: 300 designs. Exercise 9: all 300 hit y within 0.02 on average and are spread all around the circle (357°).

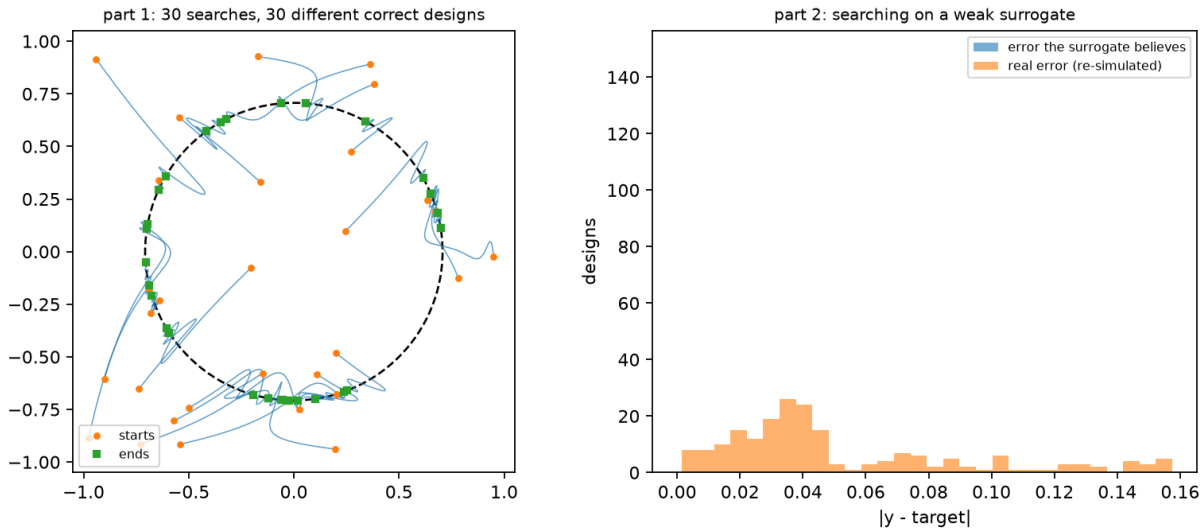
**β is a balance.** Too large and the codes carry no information, so designs ignore the target (the exercise with β = 0.5 misses by about 0.44). Too small and random codes fall in gaps and give poor designs. β depends on how the data are scaled: the project uses β = 0.5 on normalised data with a 4-number code.




*Exercise 9: 300 CVAE designs for each of three targets; dashed circles are the sets of correct designs.*

## 8.5 Multi-start gradient search

No inverse network at all. Start from 64 random designs and move each one downhill on (F(design) − target)² + λ·penalty, using the forward network's gradient — gradient descent (7.1) on the *design* instead of on the network's parameters. The design is written as u = sigmoid(w) so it can never leave the allowed box. Different starts end at different correct designs (exercise 10, part 1). Cost: a few hundred forward evaluations per start — about 0.6 s per target in the project.




*Exercise 10: left, 30 searches from random starts (orange) each end at a different correct design (green). Right, searching on a weak surrogate: it believes every design is perfect (blue), the re-simulation disagrees (orange).*

## 8.6 The trap: surrogate exploitation

The forward network is not the truth; it has small errors. An optimiser that searches hard finds exactly the places where the network is wrong *in its favour*. Exercise 10, part 2: with a weak surrogate the search reports a perfect 0.000 error, but the true error is 0.051 on average and up to 0.157. Two consequences in the project: (1) **every design is re-simulated with PWE** before it counts; (2) searching harder is not always better — the tandem, which cannot search, came out most accurate.

## 8.7 CVAE + refine

Combine the two: take the CVAE's designs as good, varied starting points and run a short gradient search (100 steps) from each. The CVAE supplies diversity, the search supplies accuracy, and starting close to an answer keeps the search short, which limits exploitation.

## 8.8 How the project compares them

1. Take 100 test cells with a gap that no model saw; their band edges are the targets.
2. Each method proposes designs; the surrogate ranks them; the exact fabrication check removes illegal ones.
3. The chosen design is **re-simulated with PWE**; the error |PWE − target| in GHz is what counts.
4. Score: share of targets hit within 1 GHz, MAE, how many *different* verified designs per target (up to 8), and time per target.

| method | within 1 GHz | MAE f_lower / f_upper (GHz) | different designs (of 8) | s per target |
|---|---|---|---|---|
| Tandem NN | 100% | 0.07 / 0.10 | 1.0 | 0.00 |
| CVAE | 92% | 0.32 / 0.37 | 5.1 | 0.00 |
| Multi-start | 94% | 0.17 / 0.34 | 7.2 | 0.59 |
| CVAE + refine | 97% | 0.09 / 0.20 | 7.9 | 0.20 |

*Project results, seed 0 (results/optionb/results.json). Repeated with 5 random splits, the share within 1 GHz is (mean ± spread): Tandem NN 99 ± 1%; CVAE 93 ± 2%; Multi-start 92 ± 3%; CVAE + refine 97 ± 1%. The ranking does not depend on luck.*

**How to read it:** the tandem is the most accurate but gives one design; the CVAE alone is varied but less accurate; multi-start is accurate and varied but slower; CVAE + refine gives nearly 8 different verified designs per target at 97% success — the best trade-off. For a thesis, 'best' depends on what the designer needs: one answer fast, or many to choose from (for example for fabrication tolerance).

**Exercise 8–10 — ex08_tandem_toy.py, ex09_cvae_toy.py, ex10_multistart_toy.py**

**Run:**

```
python learn\ex08_tandem_toy.py
python learn\ex09_cvae_toy.py
python learn\ex10_multistart_toy.py
```

**What it shows:** direct inverse vs tandem; CVAE sampling; multi-start search and surrogate exploitation.
**Then try:** in ex09 BETA = 0.5 and 0.002; in ex10 N_TRAIN = 400 (does the exploitation gap shrink?).

### Check yourself

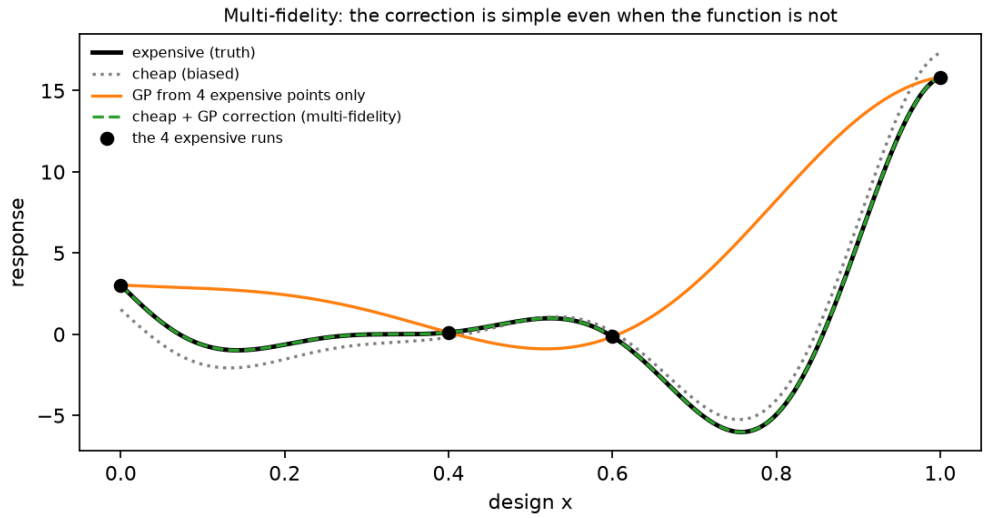
1. Why does a direct inverse network fail when many designs give the same response?
2. In the tandem, which network learns in step 2, and what is the loss?
3. What is the code z of a CVAE for?
4. What happens if β is too large?
5. What is surrogate exploitation and how does the project guard against it?
6. Why combine CVAE and gradient search?

# 9. Multi-fidelity: cheap simulations corrected by expensive ones

The project has two simulators: PWE (0.2 s, about 1 GHz off) and COMSOL (accurate, minutes). The Kennedy–O'Hagan idea: model the expensive answer as the cheap answer plus a correction δ, and learn δ with a Gaussian process from the few cells where both exist:

$$f_{\mathrm{COMSOL}}(\mathbf{x}) \approx f_{\mathrm{PWE}}(\mathbf{x}) + \delta(\mathbf{x}), \qquad \delta \sim \mathcal{GP}$$

Why it works: the cheap simulator already captures the hard, wiggly part; the difference is usually smooth and small, and a GP learns smooth things from very few points. Exercise 11 makes this concrete: with only 4 expensive points, a GP on the expensive function alone is badly wrong (error 3.99), the cheap simulator alone is off by 0.75, and cheap + learned correction is essentially exact.




*Exercise 11: four expensive runs (black dots) are not enough to learn the function (orange), but they are plenty to learn the smooth difference between cheap and expensive (green).*

In the project the correction GP uses the 18 shape features and the 157 COMSOL cells with a gap. On cells it did not see (10-fold cross-validation) it reduces the $f_{\mathrm{lower}}$ error from 0.89 to 0.09 GHz and the gap error from 0.90 to 0.29 GHz. The catch: when a whole shape family is left out and predicted from the others, the correction does not always help ($f_{\mathrm{lower}}$ GHz: circle 0.92 → 0.13; rounded 0.88 → 1.13; sharp 0.87 → 1.76). The correction depends on the shape, so it needs COMSOL cells of each new family — for freeform shapes, the gallery designs.

**Exercise 11 — ex11_multifidelity_toy.py**

**Run:**

```
python learn\ex11_multifidelity_toy.py
```

**What it shows:** cheap simulator, expensive simulator and the learned correction.

**Then try:** make the bias wigglier (add 2·sin(10x) in cheap()); move the 4 expensive points.

**Check yourself**

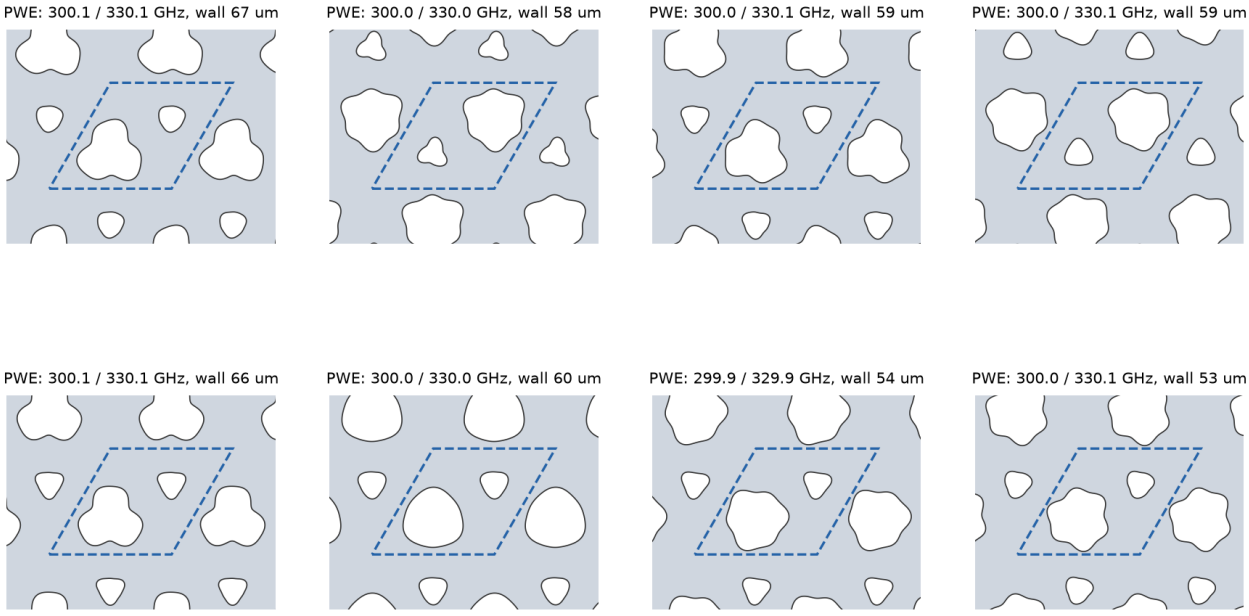
1. Write the multi-fidelity model in one line.
2. Why can the correction be learned from few points?
3. Why does the project's correction not transfer to an unseen shape family?

# 10. The whole pipeline on one example

A designer asks for a band gap from **300 to 330 GHz** (inside the 6G band). Here is everything that happens, with the part of this guide that explains each step:

1. **Data (parts 4, 6).** 6,000 random fabricable cells were described by their 18 Fourier numbers and labelled with PWE at M and K (about 4 minutes in total). The PWE solver was calibrated (ε = 8.914) and checked against all 177 COMSOL cells.
2. **Forward surrogate (7).** An ensemble of 5 networks learned 18 numbers → ($f_{lower}$, $f_{upper}$), with about 0.13 / 0.17 GHz error on unseen cells.
3. **Inverse models (8).** The tandem and the CVAE were trained using the surrogate.
4. **Proposals (8).** For the target (300, 330): 512 CVAE designs, 256 multi-start designs and 256 CVAE + refine designs.
5. **Checks (4.6, 8.6).** Every candidate is ranked by the surrogate, checked against the fabrication rules, and re-simulated with PWE; only those within 1 GHz of both targets are kept, and near-duplicates (including the same shape with A and B swapped or mirrored) are removed.
6. **Result.** 24 different verified shapes, all giving 300 / 330 GHz to within 1 GHz (option-B guide, figure 'showcase'). They are exported to `data/pwe/showcase_for_comsol.txt`.
7. **Final word (9).** Simulate some of them in COMSOL. Their COMSOL results also extend the multi-fidelity correction to freeform shapes — the active-learning loop of the pipeline figure.

Different shapes, same target band edges 300 / 330 GHz (24 distinct designs verified within 1 GHz)




*Different hole shapes, same band gap: the non-uniqueness that the inverse methods of part 8 are built to handle.*

# 11. Glossary

| term | meaning |
|---|---|
| a, a1, a2 | lattice constant (242.5 µm) and lattice vectors; the pattern repeats along them |
| unit cell | the smallest tile that repeats; here a rhombus with two holes A and B |
| permittivity ε, index n | material constants, $\varepsilon = n^2$; silicon 11.7 bulk, 8.914 fitted here |
| wavevector k | direction and phase rate of a wave, $\lvert k\rvert = 2\pi/\lambda$ |
| reciprocal lattice G | G = m b1 + n b2; the wavevectors of anything periodic in the crystal |
| Brillouin zone | hexagon of distinct wavevectors; Γ centre, M edge middle, K corner |
| band, band diagram | allowed frequencies for each k; plotted along Γ–M–K–Γ |
| band gap, f_lower, f_upper | frequencies with no allowed wave; its bottom and top edges |
| Dirac point | where bands 1 and 2 touch at K (equal holes) |
| valley photonic crystal | honeycomb crystal with unequal holes: gap at K and K', protected edge waves |
| Fourier series / coefficients | a repeating pattern written as a sum of cosines and sines / their amounts |
| FFT | fast algorithm computing all Fourier coefficients from samples |
| C3 symmetry, selection rule | unchanged by a 120° turn; allows only harmonics n = 3, 6, 9, 12 |
| Bloch wave | plane wave × a function with the crystal's period |
| PWE | plane-wave expansion: field as plane waves, Maxwell's equation as a matrix eigenproblem |
| Toeplitz matrix, Ho's rule | matrix constant along diagonals [$\varepsilon_{G-G'}$]; inverting it gives fast convergence |
| eigenvalue / eigenvector | $A\,v = \lambda\,v$; here $\lambda = (\omega/c)^2$ and v the wave's plane-wave amounts |
| TE polarisation | magnetic field out of the plane, electric field in the plane |
| COMSOL | finite-element simulator; accurate, slow; the reference (high fidelity) |
| surrogate | fast learned model that imitates a simulator |
| parameter, loss, gradient descent | adjustable numbers of a model; error measure; downhill updates |
| MLP, SiLU, Adam | layered neural network; its smooth activation; its optimiser |
| ensemble | several networks; mean = prediction, spread = uncertainty |
| Gaussian process, kernel, ARD | regression from similarity; the similarity function; one length scale per input |
| overfitting, test set | memorising instead of learning; data kept aside to measure honest errors |
| MAE | mean absolute error, always in GHz here |
| inverse design, non-uniqueness | target → design; many designs give the same target |
| tandem network | inverse net trained through a frozen forward net, loss on the response |
| CVAE, latent code z, β, KL | generative model; random code picks the design; weight of the KL term; distance between distributions |
| multi-start gradient search | many downhill searches on the design from random starts |
| surrogate exploitation | optimiser finding the surrogate's errors instead of real solutions |
| multi-fidelity correction | cheap result + GP-learned difference to the expensive one |
| seed | the number that fixes the random choices (data split, initial weights) |

# 12. Formula sheet

**Waves**

$$c = f\lambda, \quad \lambda_{\text{material}} = \lambda/n, \quad \varepsilon = n^2, \quad f[\text{GHz}] = u \cdot \tfrac{c}{a} = u \cdot 1236 \ (a = 242.5\,\mu m)$$

**Band edges**

$$f_{\text{lower}} = \max_{\mathbf{k}} f_1, \quad f_{\text{upper}} = \min_{\mathbf{k}} f_2, \quad \Delta f = f_{\text{upper}} - f_{\text{lower}}$$

**Fourier series**

$$g(x) = \sum_m c_m e^{i2\pi mx/a}, \quad c_m = \tfrac{1}{a}\int_0^a g\, e^{-i2\pi mx/a} dx, \quad c_{\pm m} = \tfrac{a_m}{2}$$

**Hole shape (C3)**

$$r(\theta) = r_0 + \sum_{n\,=\,3,\,6,\,9,\,12} [c_n \cos n\theta + s_n \sin n\theta]$$

**Layer coefficients**

$$a_0 = \varepsilon_{\text{air}} + (\varepsilon_{\text{Si}} - \varepsilon_{\text{air}})F, \quad a_m = 2(\varepsilon_{\text{Si}} - \varepsilon_{\text{air}})\tfrac{\sin \pi mF}{\pi m}$$

**PWE (1D and 2D, TE)**

$$\sum_{\mathbf{G}'} (\mathbf{k}+\mathbf{G}) \cdot (\mathbf{k}+\mathbf{G}') [\varepsilon^{-1}]_{\mathbf{G}\mathbf{G}'} h_{\mathbf{G}'} = \tfrac{\omega^2}{c^2} h_{\mathbf{G}}, \quad [\varepsilon^{-1}] = [\varepsilon_{\mathbf{G}\,-\,\mathbf{G}'}]^{-1}$$

**Reciprocal vectors**

$$\mathbf{b}_1 = \tfrac{2\pi}{a}(1, -1/\sqrt{3}), \ \mathbf{b}_2 = \tfrac{2\pi}{a}(0, 2/\sqrt{3}), \ \mathbf{a}_i \cdot \mathbf{b}_j = 2\pi\delta_{ij}$$

**Gradient descent, MSE**

$$w \leftarrow w - \eta \tfrac{\partial \mathcal{L}}{\partial w}, \qquad \mathcal{L} = \tfrac{1}{N}\sum_i (\hat{y}_i - y_i)^2$$

**Tandem and CVAE**

$$\mathcal{L}_{\text{tandem}} = |F(G(\mathbf{t})) - \mathbf{t}|^2 + \lambda P, \qquad \mathcal{L}_{\text{CVAE}} = |\hat{\mathbf{x}} - \mathbf{x}|^2 + \beta\, \tfrac{1}{2}\sum (\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$$

**Multi-fidelity**

$$f_{\text{COMSOL}} \approx f_{\text{PWE}} + \delta, \qquad \delta \sim \mathcal{GP}(0, k)$$

# 13. Answers to the check-yourself questions

**Part 2**

**Q.** What is the wavelength in air at 250 GHz?

*A.* $\lambda = c/f = 2.998\times10^{8}/2.5\times10^{11} = 1.2$ mm.

**Q.** Why does a periodic structure have band gaps?

*A.* Reflections from the many equally spaced boundaries add up in step at certain frequencies, giving total reflection.

**Q.** What do k = 1 and k = 2 mean in your COMSOL files?

*A.* The high-symmetry points M and K of the Brillouin zone.

**Q.** Define $f_{lower}$ and $f_{upper}$.

*A.* $f_{lower}$ = highest frequency of band 1 over all k; $f_{upper}$ = lowest frequency of band 2 over all k; the gap lies between them.

**Q.** If you make the whole crystal 10% larger, what happens to the gap?

*A.* All frequencies fall by a factor 1.1 (divide by 1.1).

**Q.** Why do the two holes have to be different?

*A.* Identical holes give a Dirac point (no gap) at K; different holes break the A/B symmetry and open the valley gap.

**Part 3**

**Q.** What does $a_0$ represent?

*A.* The average value of the pattern.

**Q.** Why are all $b_m$ zero for the layer pattern centred at x = 0?

*A.* The pattern is even (symmetric) and sines are odd; their product averages to zero.

**Q.** Compute $a_1$ for F = 0.25 (ε_Si = 8.914).

*A.* $a_1 = 2\cdot7.914\cdot\sin(\pi/4)/\pi = 15.828\cdot0.7071/3.1416 = 3.563$.

**Q.** How are $c_1$ and $a_1$ related for an even pattern?

*A.* $c_1 = c_{-1} = a_1/2$.

**Q.** Why do sharp edges need many Fourier terms?

*A.* A sudden jump contains fine detail, i.e. high frequencies; only high-m cosines can build it.

**Part 4**

**Q.** Which Fourier terms can a three-fold symmetric hole have, and why?

*A.* n = 3, 6, 9, ...: only then is n·120° a whole number of turns, so the term is unchanged by a 120° rotation.

**Q.** What shape is $r = 40 + 0\cdot\cos 3\theta$?

*A.* A circle of radius 40 µm.

**Q.** How do you rotate a c3 cos 3θ shape by 30°?

*A.* Use s3 sin 3θ instead (or a mix of c3 and s3).

**Q.** Why is c3 of the old rounded triangle negative?

*A.* θ = 0 points at a flat side, where r is smallest; a negative cos 3θ term is smallest at θ = 0.

**Q.** How many input numbers do the models get, and why that many?

*A.* 18: 9 per hole (r0 plus cosine and sine of n = 3, 6, 9, 12) for 2 holes.

**Part 5**

**Q.** What does Bloch's theorem say?

*A.* A wave in a periodic crystal is a plane wave $e^{ikx}$ times a function with the period of the crystal.

**Q.** Why does a derivative become a multiplication in PWE?

*A.* d/dx of $e^{iqx}$ is $iq \cdot e^{iqx}$: each plane wave is only multiplied by iq.

**Q.** Where do the matrix's off-diagonal entries come from?

*A.* From the Fourier coefficients of 1/ε ($\eta_{m-m'}$): the pattern couples plane waves whose G differ by $G_{m-m'}$.

**Q.** What do the eigenvalues of the PWE matrix mean?

*A.* $(\omega/c)^2$ of the allowed waves at that k; take the square root and divide by 2π (times c/a) to get frequencies.

**Q.** What is Ho's rule and why use it?

*A.* Invert the Toeplitz matrix of ε's coefficients instead of using 1/ε's coefficients; it converges much faster for sharp material boundaries.

**Q.** With u = 0.25 and a = 242.5 µm, what is the frequency?

*A.* f = u·c/a = 0.25·1236 GHz = 309 GHz.

**Part 6**

**Q.** What is a reciprocal lattice vector?

*A.* A vector G = m b1 + n b2 with $b_i \cdot a_j = 2\pi\delta_{ij}$; the Fourier series of anything periodic in the crystal uses exactly these wavevectors.

**Q.** Where does the hole shape enter the PWE solver?

*A.* When building the ε picture: a sub-pixel is air if its distance to the hole centre is smaller than r(θ).

**Q.** Why average 16 sub-pixels per pixel?

*A.* So ε changes smoothly when the shape changes; frequencies then change smoothly too, which gradient-based design needs.

**Q.** What is computed once per design and what once per k-point?

*A.* The inverse ε matrix once per design; the eigenvalue problem once per k-point.

**Q.** Why was ε fitted?

*A.* Bulk-silicon ε made every frequency 13% low by the same factor; fitting one number (8.914) matched COMSOL for all cells, consistent with an effective index in the 2D model.

**Part 7**

**Q.** What is a loss function?

*A.* A number measuring how wrong the model is on the examples, e.g. the mean squared error; training minimises it.

**Q.** Do one more gradient-descent step of the table in 7.1.

*A.* w = 1.875, slope = 5·1.875 − 10 = −0.625, new w = 1.875 + 0.0625 = 1.9375.

**Q.** Why keep a test set aside?

*A.* To measure the error on cells the model never saw; errors on training cells hide overfitting.

**Q.** What does the spread of an ensemble tell you?

*A.* How uncertain the prediction is: large spread = the networks disagree = little data nearby.

**Q.** When would you prefer a GP over a neural network?

*A.* With few examples (tens to a few hundred) and when you want a calibrated uncertainty.

**Q.** Why does the project's network have SiLU between layers?

*A.* Without a non-linear activation the stacked layers collapse into one linear formula, which cannot fit curved relations.

**Part 8**

**Q.** Why does a direct inverse network fail when many designs give the same response?

*A.* Minimising squared error makes it predict the average of all correct designs, which is usually not a correct design.

**Q.** In the tandem, which network learns in step 2, and what is the loss?

*A.* Only the inverse network; the loss is (forward(inverse(target)) − target)² plus the fabrication penalty.

**Q.** What is the code z of a CVAE for?

*A.* It selects which of the many correct designs is produced; random codes give different designs for the same target.

**Q.** What happens if β is too large?

*A.* The KL term dominates, the codes carry no information about the design, and designs stop matching the target well.

**Q.** What is surrogate exploitation and how does the project guard against it?

*A.* Optimisers find designs where the surrogate's error flatters them; every design is re-simulated with PWE before it is counted.

**Q.** Why combine CVAE and gradient search?

*A.* The CVAE gives varied starting points, the short search makes each accurate; together accurate AND diverse.

**Part 9**

**Q.** Write the multi-fidelity model in one line.

*A.* expensive(x) ≈ cheap(x) + δ(x), with δ learned by a GP from the cells where both exist.

**Q.** Why can the correction be learned from few points?

*A.* The difference between the simulators is smooth and simple even when the function itself is not.

**Q.** Why does the project's correction not transfer to an unseen shape family?

*A.* The PWE–COMSOL difference depends on the hole shape; a family it never saw can have a different bias.

## Where to read more


- J. D. Joannopoulos, S. G. Johnson, J. N. Winn, R. D. Meade, *Photonic Crystals: Molding the Flow of Light*, 2nd ed. (2008), free at ab-initio.mit.edu/book — chapters 2–5 (waves, symmetry, band gaps) and appendix D (numerical methods including plane waves).
- K. M. Ho, C. T. Chan, C. M. Soukoulis, Phys. Rev. Lett. 65, 3152 (1990) — the inverse-matrix rule.
- D. Liu et al., ACS Photonics 5, 1365 (2018) — the tandem network.
- K. Sohn, H. Lee, X. Yan, NeurIPS (2015) — conditional VAE.

- C. E. Rasmussen, C. K. I. Williams, *Gaussian Processes for Machine Learning* (2006), free at gaussianprocess.org/gpml — chapter 2.
- M. C. Kennedy, A. O'Hagan, Biometrika 87, 1 (2000) — multi-fidelity modelling.
- The option-B study guide (docs/OptionB_Study_Guide.pdf) — the project's results and the COMSOL side.